

自适应记忆在编程任务中的优化

实现代码与初步测试结果 · 题目三 C · 2026-09-08

核心结论：当前方案已在长对话留出集上同时改善质量、召回与 token 成本；编程侧已完成 linked retrieval、延迟反馈、影子学习、晋升/回退和可执行 SWE 验证链路，但尚未证明历史记忆能稳定提高 SWE-bench 最终解决率。

+0.02781 Token F1 vs Top-10	+0.03952 Evidence recall vs Top-10	23.28% 输入 token 节省率	1 / 5 SWE official resolved
---------------------------------------	--	-------------------------------	---------------------------------------

本次交付

材料	内容	状态
实现代码	AdaptiveRecallPolicy、CodingFeedback、PromotionGate、AutoRecall 接入	已实现并测试
实验链路	SWE search/read/test/edit、复现门、窄回归、失败分类与回滚	可运行
初步结果	LoCoMo 独立对照、5 题 SWE 结果、promotion smoke	已固化
研究边界	不把机制通过写成成功率提升，不包含 gold patch 或 hidden tests	已声明

开源仓库：<https://github.com/acidmelon/TencentDB-Agent-Memory/tree/topic3c-adaptive-memory>
固定实现提交：41d028064a199357c5efc94c4d976c1c9bc98b43
基线：TencentDB Agent Memory v2.0.0-beta.1（4144434）。仓库由 GitHub Fork 流程创建，可直接核对上游来源。

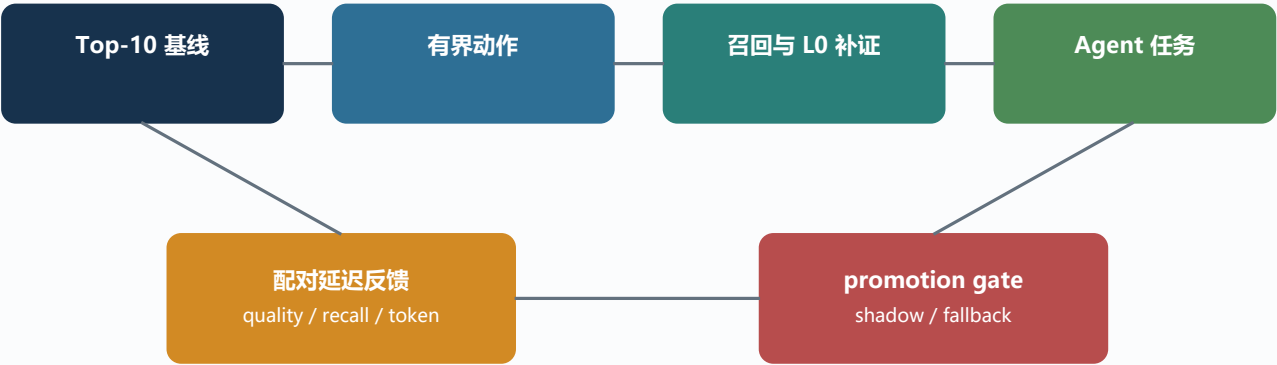
1. 问题与方案定位

原系统以固定 Top-10 召回为主。固定缩小 K 可以节约上下文，却可能损失关键证据；直接增加搜索或反复修复又会显著增加 token。编程任务还存在更严格的问题：一次补丁失败可能来自定位错误、过时证据、编译错误或回归，不能仅用语言相似度作为学习信号。

研究问题

能否在不重写 FTS5、向量检索和 RRF 的前提下，根据已完成任务的延迟反馈，自适应选择更合适的记忆组织方式，并在风险出现时可靠回退？

运行闭环



任何 warmup、状态损坏、L0 失败或硬回归均回到 Top-10

设计不变量

不变量	实现方式
Top-10 始终可用	warmup / shadow / 失败路径强制使用基线
动作空间有界	只允许 top10、top8、top5、top5-l0
收益可归因	动作与同任务 Top-10 结果成对记录
质量与成本分离	quality、recall、token 分别建模
项目状态隔离	team / agent / project 组成学习作用域；缺省项目显式落到 default-project

2. 实现与代码质量

模块	职责	关键安全属性
adaptive-policy.ts	经验均值 / LinUCB、持久化状态、有限动作	原子 rename、每作用域更新锁、TTL、损坏回退
coding-feedback.ts	将 compile、tests、regression、pass@1 转为配对质量	字段校验；不以 token 代替成功
promotion-gate.ts	质量/成本下界、覆盖率、晋升和硬降级	学习与启用分离；当前在实验适配/回放层
auto-recall.ts	取得策略决策并覆盖 maxResults；可补 L0 证据	L0 失败保留 L1；默认关闭
swe-agent-v2.py	linked 定位、复现、编辑、窄回归、回滚	实验求解器；不读取 evaluator-only 信息

提交前审查结论

维度	结论
与上游风格一致性	沿用 TypeScript ESM、typed interface、现有 hook 目录和日志方式；Python 实验代码与生产路径隔离。
注释	主要解释安全锚点、反馈语义与 fallback 原因；没有为显然语句堆叠注释。
测试	3 个 TypeScript 文件 17 项、Python 14 项、插件构建及 promotion smoke 均通过。
已修问题	AutoRecall 现会把调用方提供的 projectId 传入策略作用域；容忍阈值注释明确为 delta 下界。
刻意不做的重构	adaptive-policy.ts 约 458 行，后续可拆持久化/统计/学习器；中期提交前拆分会扩大回归面。

质量判断：达到可审查、可运行的中期实现标准。当前最重要的改进不是增加注释数量，而是扩大独立任务验证，并让上层入口提供真实 projectId。

3. 探索实验与方案演化

探索实验的作用是解释当前设计从何而来，而不是把所有临时产物放进交付仓库。下面只保留改变方案选择的结果。

阶段	主要观察	最终处理
固定 Top-5 / 8	节省 token，但查询间质量变化不一致	保留为动作，不全局替换
L1 + L0 邻居	可补原始上下文，成本可控	形成 top5-l0；失败保留 L1
KNN / 分桶	小样本容易受局部噪声影响	保留稳定特征与收缩思想
LinUCB	无需额外 LLM selector，可表达上下文差异	与 mean 并列；受 warmup 和门限保护
函数卡片	有助定位，但自由文本可能陈旧或误导	改为可回到当前源码核验的 linked 线索
即时记忆	容易固化偶然错误	改为编译/测试后的延迟配对反馈
Reproduction + repair	多 33,626 token，5 题解决数仍为 1	保留验证纪律，不把更多轮次当收益

当前工程候选

Top-10 安全锚点 + 有界自适应动作 + 同任务配对延迟反馈 + 按项目作用域的晋升/回退；编程侧叠加 linked retrieval、test-first reproduction、窄回归和失败回滚。

为何暂不直接注入函数用途卡片

函数名、用途和参数可由同一 LLM 抽取，也可以由 AST/索引器确定性生成。但如果卡片脱离当前提交，它可能把旧签名或错误解释直接放进上下文。现阶段更稳妥的做法是把函数信息作为检索线索，附带文件与符号位置，让模型回到当前源码核验；待建立版本绑定和陈旧检测后，再评估结构化函数卡片。

选择标准

当前方案不是被宣称为 SWE 成功率的全局最优，而是在质量、成本、安全性、可归因性和实现复杂度之间最合理的候选。完整演化记录见 docs/topic3c/EXPERIMENT_HISTORY.md。

4. 长对话场景：已有正向独立对照

冻结策略在 5 个未参与方法开发的 LoCoMo 对话上验证。每个对话前 24 条用于 onboarding，之后共评价 923 条 query；比较对象是同一输入下的 Top-10。



说明：前三项量纲不同，条形仅展示各指标相对自身坐标上限，不用于指标间大小比较。

指标	Adaptive	相对 Top-10	95% CI (query / dialog cluster)
Token F1	0.21458	+0.02781	[+0.01427,+0.04153] / [+0.01851,+0.03546]
Evidence recall	0.36380	+0.03952	[+0.02046,+0.05896] / [+0.00964,+0.05881]
平均输入 token	209.50	-63.57	[-67.28,-59.89] / [-73.14,-55.14]
Token 节省率	-	23.28%	由同一批配对输入计算

可支持的结论

在这 5 个独立对话、923 条评估 query 的固定协议下，自适应组织相对 Top-10 同时提高 F1 与 evidence recall，并减少输入 token；两类 bootstrap 区间均不跨 0。

边界

独立单位只有 5 个对话，且 LoCoMo 是公开长对话代理任务，不是软件仓库任务。它证明自适应记忆机制可以产生质量/成本收益，但不能直接推导 SWE-bench 成功率提升。

5. 编程场景：机制成立，成功率收益待验证

实验	公开复现	Official resolved	API token	结论
5 题 public reproduction	5 / 5	1 / 5	114,384	基础求解链路
reproduction + repair	5 / 5	1 / 5	148,010	多 33,626 token，未增加解决数
Matplotlib 13989	通过	未单列	-	linked 定位 + 窄回归得到公开验证补丁
Astropy 14096 adaptive	记忆注入成功	无有效补丁	-	触发成功不等于任务成功

Promotion smoke

阶段	观察	系统行为
任务 1-23	样本不足	onboarding / Top-10
任务 24	质量、成本与编程覆盖率通过	promoted
硬回归任务	regression + stale evidence	立即 demoted / Top-10

Smoke 只证明门控状态机和回退机制，不是效果数据。PromotionGate 当前由实验适配/回放层调用，尚未默认串入 AutoRecall 线上路径。

目前能确认

候选源码组织可改善部分任务的目标定位；asserted reproduction、窄测试、失败分类和回滚可减少无效验收；编译、测试和回归反馈能进入自适应策略。

目前不能确认

历史 issue 记忆能稳定提高 SWE-bench resolved；函数用途卡片优于 linked 源码；当前少量开发题能够代表跨项目泛化。

6. 复现、交付与下一阶段

仓库结构

路径	用途
MemoryCore/src/core/hooks/	自适应策略、编程反馈、晋升门、AutoRecall 接入与单测
MemoryCore/scripts/topic3-c/	反馈更新、回放、SWE 适配器与实验求解器
docs/topic3c/	实现说明、结果、探索演化与报告生成器
results/	精炼后的机器可读初步结果和 promotion smoke

最小复现

环境	命令	已记录结果
Node >=22.16	cd MemoryCore && npm install	依赖准备
TypeScript	npm run test:topic3c	3 files / 17 passed
Build	npm run build:plugin	passed
Smoke	npm run smoke:topic3c	task 24 promoted; hard failure demoted
Python	python -m pytest -q scripts/topic3-c/test_swe_agent_v2.py	14 passed

下一阶段实验

冻结模型、预算与基线，在更多独立仓库任务上比较 no-memory、Top-10 和候选策略；按项目报告 resolved、公开测试、回归、token 与失败类型。只有跨项目重复出现收益后，才把 promotion gate 接入默认 AutoRecall。函数卡片则应先补版本绑定、AST 校验和陈旧检测，再做独立消融。

中期结论：已经交付可运行的核心机制和诚实的初步结果。长对话场景观察到正向质量/成本收益；编程场景的主要贡献目前是更可控、可验证、可回退的记忆优化闭环，而非已证实的成功率提升。

。

仓库分支：<https://github.com/acidmelon/TencentDB-Agent-Memory/tree/topic3c-adaptive-memory>

固定实现：41d028064a199357c5efc94c4d976c1c9bc98b43

安全说明：公开仓库不包含 API key、模型缓存、Docker 镜像、完整 benchmark bundle、gold patch 或 hidden tests。